TEST C AVANZADO EN LINUX

1) **Analiza este código:**

```
#pragma pack(push, 1)
typedef struct {
    uint8_t type;
    uint32_t length;
} header_t;
#pragma pack(pop)
```

¿Cuál es la afirmación correcta?

a) sizeof(header_t) debe ser 8 y length estará alineado a 4 bytes
b) sizeof(header_t) debe ser 5; length comienza en el offset 1 y puede quedar desalineado
c) La directiva convierte automáticamente length a orden de red
d) El resultado depende únicamente de si la CPU es little-endian

2) **Analiza el siguiente código:**

```
int *values = malloc(10 * sizeof(*values));
if (values == NULL) { return 1; }
values = realloc(values, 100 * sizeof(*values));
if (values == NULL) { return 1; }
```

¿Qué sucede si realloc no puede ampliar el bloque?

a) realloc libera el bloque original y devuelve NULL, por lo que no hay fuga
b) El bloque original sigue reservado, pero su dirección se pierde al sobrescribir values
c) realloc conserva siempre la misma dirección y reduce el array a cero elementos
d) El comportamiento es indefinido porque realloc no permite ampliar bloques ya reservados

3) **Un cliente TCP ejecuta estas dos llamadas consecutivas:**
send(fd, "ABC", 3, 0);
send(fd, "DEF", 3, 0);

El servidor dispone de un buffer suficientemente grande y llama a recv().
¿Cuál de las siguientes afirmaciones es correcta?

a) El servidor recibirá necesariamente primero ABC y después DEF en dos llamadas distintas a recv()
b) El servidor recibirá necesariamente ABCDEF en una única llamada a recv()
c) El servidor puede recibir ABCDEF de una vez o dividido en varias lecturas, aunque el orden de los bytes se conserva
d) El servidor puede recibir DEF antes que ABC porque TCP no garantiza el orden

4) **Un servidor TCP utiliza sockets no bloqueantes con epoll en modo edge-triggered:**
struct epoll_event event;
event.events = EPOLLIN | EPOLLET;
event.data.fd = client_fd;
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &event);

Cuando epoll_wait informa de que client_fd está preparado, el servidor ejecuta una única llamada: ssize_t n = recv(client_fd, buffer, sizeof(buffer), 0); Aunque recv devuelve datos, todavía quedan bytes pendientes en el buffer de recepción del socket.
¿Cuál es el problema principal?

a) Ninguno: epoll_wait notificará inmediatamente otra vez mientras queden bytes pendientes
b) El servidor puede no recibir otra notificación para esos bytes, porque con EPOLLET debe leer repetidamente hasta que recv devuelva -1 con EAGAIN o EWOULDBLOCK
c) recv debe sustituirse siempre por readv cuando se utiliza EPOLLET
d) EPOLLET solo es válido para sockets UDP, no para conexiones TCP

5) **¿Cuál es la afirmación correcta sobre pthread_detach() en un programa multihilo?**
   a) Desvincula el hilo del proceso, permitiéndole continuar, aunque main retorne
   b) Hace que los recursos del hilo se liberen automáticamente cuando termina, pero impide recogerlo posteriormente con pthread_join()
   c) Finaliza inmediatamente el hilo indicado y libera todos sus recursos
   d) Convierte el hilo en un proceso independiente con su propio espacio de memoria

6) **¿Cuál es la diferencia fundamental entre un socket TCP y un socket UDP respecto al tratamiento de los datos enviados?**
   a) TCP conserva los límites de cada mensaje, mientras que UDP entrega un flujo continuo de bytes
   b) TCP y UDP garantizan la entrega y el orden; únicamente cambia la velocidad
   c) UDP requiere establecer una conexión con accept(), mientras que TCP no necesita conexión
   d) TCP proporciona un flujo de bytes fiable y ordenado, mientras que UDP conserva los límites de los datagramas sin garantizar necesariamente su entrega ni su orden

7) **Analiza el siguiente código:**

```
#include <stdio.h>
static void mostrar(const void *dato) {
        const int *numero = dato;
        printf("%d\n", *numero);
}
int main(void) {
        double valor = 3.5;
        mostrar(&valor);
return 0;
 }
```

¿Cuál es la afirmación correcta?

a) El programa imprime 3 porque la conversión de double a int se realiza automáticamente
b) El programa es correcto porque cualquier objeto puede desreferenciarse directamente mediante void *
c) La conversión del puntero es válida, pero desreferenciarlo como int cuando apunta a un double produce comportamiento indefinido
d) El compilador rechazará obligatoriamente la llamada mostrar(&valor) porque void * solo admite direcciones de int

8) ¿Cuál es la afirmación correcta sobre fork() en un programa Unix/Linux?

a) fork() crea un hilo que comparte la pila y todas las variables locales con el proceso padre
b) fork() sustituye inmediatamente el proceso actual por otro programa
c) fork() crea un proceso hijo con un espacio de memoria independiente; ambos continúan desde la instrucción posterior a fork() y reciben valores de retorno diferentes
d) fork() devuelve siempre el PID del hijo tanto en el proceso padre como en el hijo

9) Un programa falla de manera intermitente porque una variable cambia a un valor incorrecto, pero no se conoce qué instrucción la modifica. ¿Qué recurso de GDB es el más adecuado para detener la ejecución exactamente cuándo cambia esa variable?

a) Un breakpoint normal situado en main()
b) Un watchpoint sobre la variable
c) El comando backtrace antes de ejecutar el programa
d) El modo TUI con layout asm

10) **Al ejecutar un programa con Valgrind Memcheck aparece este resumen: HEAP SUMMARY: in use at exit: 256 bytes in 1 blocks LEAK SUMMARY: definitely lost: 256 bytes in 1 blocks indirectly lost: 0 bytes in 0 blocks possibly lost: 0 bytes in 0 blocks still reachable: 0 bytes in 0 blocks**

¿Qué conclusión es correcta?

a) El programa ha perdido todos los punteros válidos al bloque de 256 bytes, por lo que existe una fuga de memoria confirmada
b) El bloque sigue siendo accesible mediante un puntero válido y no representa una fuga real
c) Valgrind indica una lectura fuera de límites de exactamente 256 bytes
d) El bloque fue liberado dos veces, pero Valgrind pudo recuperarlo automáticamente